

Facilitator answer key — Holocron capstone worked solution. Facilitator-only — lives in the facilitator folder of every repo; do not hand to participants. Part of the internally-consistent set in this folder (see `README.md`).

PRD-light — Holocron (Release-1 slice)

Persona: Content Owner — the business/content associate who owns the UI text for one or more enterprise applications (labels, buttons, errors, help text, legal notices). **Job-to-be-done:** "Fix or publish a piece of UI text — including its compliance-designated strings — myself, in minutes, without opening an engineering ticket or waiting for a release." **Why it's hard today:** "A one-word fix to a legal disclaimer is a code change, a review, and a release cycle. Work that should take minutes takes days or weeks — and there's no single record of who approved what."

Top-3 pain rank (Severity × Frequency × Addressability): (1) every text change needs an engineer + deploy — high on all three, survives; (2) no durable audit record of compliance approvals — high severity, lower frequency; (3) translation quality varies by team — high frequency, lower addressability in a 4-day slice. Pain #1 anchors this PRD; #2 and #3 are carried as governance and translation requirements.

1. Context (½ page)

Strategic anchor. Leadership's mandate: a *centralized, governed way to manage UI text* so content owners can create, review, and publish text without hardcoding it or redeploying applications — while preserving traceability, clear ownership, and approval controls. Holocron is that platform.

Scale (the numbers that shape every decision). 150+ countries, 20–30+ consuming applications, 30–40 languages/locales, `en-US` as the authoritative source, ~500,000

strings, Internal data classification. Enterprise identity (Entra ID) already covers authentication and role mapping; hosting is enterprise Kubernetes (AKS).

Data point from the brief. "Changing a sentence requires an engineer and a deployment" — a one-word correction to a legal disclaimer becomes a ticket, a code change, a review, and a release cycle. Engineering capacity is consumed by text edits that carry no engineering value, and compliance-sensitive strings can ship without a consistent review gate or a durable, centralized record of who approved what and when. That is a live audit exposure, not a nice-to-have.

2. Problem (½ page)

The Content Owner cannot change the words their product shows to users. Every edit — a typo, a clarified error message, a seasonal reword, a corrected legal disclaimer — is blocked behind engineering and a deployment. Three things follow:

- **Delivery drag.** Copy stays wrong longer than anyone wants because the fix competes for engineering capacity it doesn't need.
- **Governance risk.** Compliance-sensitive strings move through releases without a consistent legal/compliance gate, and approval evidence is scattered across teams — there is no single, durable, tamper-evident record.
- **Global inconsistency.** Decentralized ownership and per-team translation processes produce uneven quality across 30–40 languages, with `en-US` drift no one can see.

The Content Owner needs to author, govern, and publish text themselves — and consuming apps need to render the published result reliably — without touching application code.

3. Goals + non-goals (½ page)

Goals (user outcomes, not features):

1. A Content Owner can correct or publish a live UI string in minutes, with no engineering ticket and no deploy.
2. Every content and approval action leaves a durable, tamper-evident record, so a compliance reviewer can always answer "who approved what, when."

3. A consuming application always renders published, locale-appropriate text — and never a blank — falling back to `en-US` transparently when a translation is missing.

Non-goals (this release):

1. We will **not** automate translation or route through the vendor exchange in-app — translation *variants and lifecycle* are tracked, but AI translation and export/import exchange are out.
2. We will **not** build cross-product aliases, advanced reviewer configuration, or rollback — governance for release 1 is single-product ownership plus optional review with audited override.
3. We will **not** deliver Figma integration, per-locale screenshot preview, or any public/unauthenticated delivery.

4. Scope

In (Release-1 slice)	Out (deferred non-goals)
Role + scope access enforcement via Entra ID (CS1)	Aliases / cross-product shared strings (CS9)
Product/app setup, unique identifier, namespace root, target-language config (CS2)	Advanced reviewer pools & eligibility config (CS10)
Create / edit source strings with version history (CS3–4)	Export/import translation exchange (CS16–18)
Publish source string, with confirm-without-review warning (CS3–4)	Source-string rollback (CS8)
Immutable version history + audit trail (CS5)	AI-assisted translation
Search & filter across keys/content/metadata (CS6)	Figma integration; per-locale screenshot preview
Governed delivery to consuming apps, published-only + fallback (CS7)	Emailed-link review UX for approvers
Request + complete review; audited publish-time override (CS11–12)	Storing binary assets / screenshots
Translation variants + lifecycle states, <code>en-US</code> fallback (CS15)	RTL scope decision (open — routed to Architecture)

Immutable namespaced keys, `en-US` as authoritative source, and audit-immutability are treated as invariants across the whole slice, not optional features.

5. Solution sketch (½ page)

Management application (web). A Content Owner signs in via Entra ID and lands on the products they own. They create a product (unique app identifier → namespace root), configure target languages, then create a string: a dotted namespaced key (immutable once saved), `en-US` source text, optional context notes, and optional Legal/Compliance/Peer review designations. Editing produces a new Draft version; prior versions are

preserved. **Publish** is explicit: if no review is designated (or a designated review is unresolved), the system shows a confirm-without-review warning; confirming publishes and writes the action — including any override and its reason — to the audit trail.

Governance surface. Reviews are optional-by-default. A designated Legal/Compliance review that is Pending or Rejected blocks a silent publish; the Content Owner must cancel or explicitly override with a reason. Every version, state transition, and review decision is immutable and reverse-chronological, filterable by actor, change type, language, and date.

Delivery surface. A consuming app authenticates and requests all strings for a namespace prefix. It receives **only Published** strings (key, content, language, version) for the requested locale — the selected translation, or `en-US` with an explicit fallback indicator when the locale is missing. Reads are served from a Redis cache; a published change propagates within $\leq 60s$ (we accept $\leq 60s$ staleness as a deliberate trade-off for delivery latency and availability).

Translation surface. For each product target language, one locale record per string tracks a Translation State (Not Translated → Awaiting Translation → Pending Review → Approved → Published → Outdated). Translations are versioned independently and pinned to the `en-US` source version they were made from; republishing the source flips stale translations to Outdated.

Passes the engineer's first three questions: What's the delivery contract? Namespace-prefix fetch, published-only, fallback-flagged. What's authoritative? `en-US`, immutable keys. What's the consistency model? Redis read-cache, $\leq 60s$ propagation.

6. Acceptance Criteria (7 total)

Happy path

1. **Create + publish with audit.** *Given* I am a Content Owner with a valid unique namespaced key and `en-US` text and no review is designated, *when* I create the string and confirm the publish-without-review warning, *then* it is saved as Published, becomes available to consuming apps, and the create and publish actions are recorded in the immutable audit trail with my identity and a timestamp.

2. **Delivery returns published + fallback.** *Given a namespace has strings in mixed lifecycle states and locale `fr-FR` has no Approved/Published translation for one string, when an authenticated consuming app requests all strings for that namespace at `fr-FR`, then only Published content is returned, the untranslated string is served as `en-US` with an explicit fallback indicator, and no Draft or Retired content appears.*
3. **Translation lifecycle transition.** *Given an `fr-FR` translation is Published against `en-US` source version 3, when I publish `en-US` source version 4, then that `fr-FR` translation transitions to Outdated, its Source Version Reference still points to v3, and no other locale's state changes.*

Sad path

4. **Publish blocked without an active owner.** *Given a product whose only Content Owner has been deactivated, when anyone attempts to publish a string in that product, then publication is blocked and the system states that an active Content Owner must be assigned before publishing can proceed.*
5. **Compliance override is audited.** *Given a string carries a Compliance designation whose review is Pending or Rejected, when I attempt to publish, then I must either cancel or explicitly override with a reason; and when I confirm the override, then it publishes and the override decision, reason, the review request/outcome, my identity, and a timestamp are recorded in the audit trail.*

Weird path

6. **Concurrent edit conflict.** *Given two Content Owners open the same string at version 5, when both save edits, then the system applies optimistic locking, accepts the first save as version 6, and presents the second user a conflict dialog showing both versions before allowing save — no silent overwrite occurs.*
7. **Key immutability.** *Given a Published string with key `checkout.button.submit`, when I attempt to rename its key, then the system prevents it and directs me to create a new string, so that consuming apps referencing the original key do not break.*

7. NFRs (top 5 — one per category)

Each: **requirement / defense / verification.**

- **Performance.** *Requirement:* namespace-scoped delivery fetch $p95 \leq 200\text{ms}$, $p99 \leq 500\text{ms}$; a publish reaches delivery within $\leq 60\text{s}$. *Defense:* consuming-app render performance across 20–30+ apps cannot wait on the management DB; a Redis read-cache serves delivery and we accept $\leq 60\text{s}$ staleness as the trade-off. *Verification:* load test against the delivery endpoint at target RPS; assert $p95/p99$ and propagation $\leq 60\text{s}$; per-consumer rate limit 50 req/s (burst 100) enforced.
- **Security.** *Requirement:* all management and delivery access is authenticated via enterprise identity (Entra ID) and authorized by role + product/locale scope; unauthenticated delivery is denied and returns no content. *Defense:* the mandate requires scoped authorization and there is no public access to text; identity is a named dependency owned by Enterprise Infrastructure. *Verification:* authz test matrix per role (Authenticated Employee / Content Owner / Reviewer / Review Admin / Holocron Admin); an unauthenticated delivery request returns denied with an empty body.
- **Accessibility.** *Requirement:* the management application meets WCAG 2.1 AA (keyboard-operable string table, editor, and review actions; visible focus; contrast). *Defense:* Content Owners across 150+ countries must operate the tool unaided; AA is the enterprise floor for internal tools. *Verification:* automated axe scan on core screens plus a manual keyboard-only pass of the create→publish→review flow.
- **Observability.** *Requirement:* emit the leading indicators from the outcome map — publish→delivery propagation latency, delivery fetch latency ($p95/p99$), delivery availability, cache staleness, and fallback-served rate per locale. *Defense:* the release-1 outcome ("published text reaches apps, correctly, within SLO") is invisible without these; fallback rate is the early signal of translation-coverage gaps. *Verification:* dashboards + alerts on each SLO breach; synthetic publish probe confirms end-to-end propagation $\leq 60\text{s}$.

- **Compliance.** *Requirement:* every content change, state transition, and review decision (including overrides) is recorded in an immutable, append-only audit trail with actor, timestamp, and before/after where applicable, retained per enterprise data-retention policy. *Defense:* the brief names durable, centralized approval evidence as a live audit exposure; Internal data classification and encryption in transit and at rest apply. *Verification:* attempt to mutate or delete an audit record (must fail); confirm override-with-reason entries and reverse-chronological filtering by actor/change-type/language/date. *Open dependency:* exact audit-retention period is unconfirmed — routed to Legal/Compliance, assumed ≥ 7 years pending confirmation.